

The analyst's first stop for any indicator.

A free, public, static threat-intelligence console for a T1/T2 SOC analyst. Paste an IP, hash, URL, domain, CVE or email and get one screen back: a verdict grounded in public reputation and vulnerability data, a ticket-ready escalation write-up, and one-click pivots to the services worth checking. Zero infrastructure, zero cost, zero accounts.

Positioning. The heavyweight open-source CTI platforms (OpenCTI, MISP, IntelOwl) need servers and databases. SOCDesk sits deliberately beneath them: the zero-infra, public-URL, analyst-speed layer an analyst actually opens mid-shift.

OPERATING PRINCIPLES — CONSTANT ACROSS EVERY LEG

- **\$0, free-tier only.** No paid infra, no paid APIs beyond free community tiers.
- **No-infrastructure.** Static site + Cloudflare Pages Functions. No database, no server to run.
- **Aggregator, not mirror.** Publish only clearly-redistributable data; reputation corpora are reached by user-clicked links, never held.
- **No secrets in the repo.** Collectors are keyless; enrichment keys live server-side in Cloudflare, never in git or the browser.
- **Privacy by default.** Analyst state (reviewed, watchlist, notable) is localStorage only. No accounts, no analytics, no tracking.
- **Strict CSP + defence in depth.** `default-src 'none'`, no inline script; every upstream string sanitised at collection and escaped again at render.
- **RADAR visual direction.** Dark warm-espresso / warm-paper, search-first, **periwinkle** accent + SD monogram; distinctive, not AI-slop. Now rebuilt on **Vite + React + Tailwind + Motion** (strict CSP kept) for an AAA craft ceiling — the live site still serves the vanilla build; the React build is staged for deploy. (Chart Room superseded.)

LEG 1 · CURRENT SPRINT

01 INDICATOR LOOKUP & ESCALATION

CORE · LIVE

The 99% workflow: paste an indicator → live reputation verdict → screenshot-grade evidence card → paste into the escalation email. This Leg is the product.

INPUT & DETECTION

- ✓ **Omnibox** with type auto-detect + refang
(evil[.]com, hxxp)
- ✓ **Bulk lookup** — many indicators, one table, CSV/JSON/defanged export
- ✓ **Bookmarklet** — fire a lookup from any page
- ✓ **Shareable #q= deep links** — "look at this" is a URL

LIVE ENRICHMENT · /API/ENRICH

- ✓ **IP** → AbuseIPDB · VirusTotal · GreyNoise · ipinfo geo/ASN
- ✓ **Hash** → VirusTotal · MalwareBazaar
- ✓ **URL / domain** → VirusTotal · urlscan verdict + screenshot
- ✓ **Consensus tally** — "N of M sources flagged"; SOCDesk counts, never asserts a verdict

OUTPUT — THE DIFFERENTIATOR

- ✓ **Escalation card** — copied into the ticket as a PNG image (Copy card) or text; no SOCDesk recommendation (the analyst's call)
- ✓ **Type-aware pivots**, trimmed to the T1 workflow set (no fluff vendors)
- ✓ **CVE** — authoritative, attributed: CISA-confirmed exploited (KEV) + CVSS + EPSS
- ✓ **Honest states** — "0 of M flagged" is never dressed as a clearance

REACH — THE RECORDED-FUTURE MOVE

- ✓ **Browser extension** — own React/MV3 build reusing the web escalation cards via a shared package; right-click lookup + popup over the live API. Final parity QA (load unpacked → context-menu handoff) pending
- **v2 · inline page annotation** — highlight indicators on any page (broad perms — later)
- **Bookmarklet selection-capture** — no-install fallback

LIVE at socdesk.io (vanilla build). Built in-tree, staged for deploy: the AAA React rebuild (web/ — Overview, Lookup, Data Desk, Gallery routes; three.js globe hero with a real light mode; per-entity escalation cards) and the extension parity uplift (shared card package, own MV3 build). Deploy flip site/ → web/ is gated. Verdict output is the consensus tally — see docs/VERDICT-LANGUAGE.md.

02 THREAT INTELLIGENCE FEED

CORE · EVOLVE, DON'T REBUILD

Situational awareness for the shift: what's happening, what's exploited, what's relevant to us. The pipeline is already strong and tested — Leg 2 is additive, not a rewrite.

ALREADY BUILT (THE FOUNDATION)

- ✓ **Collector pipeline** — KEV, NVD, EPSS, Ransomware.live, RSS pool; fault-isolated, schema-gated with last-known-good
- ✓ **CVE triage** — full KEV catalogue joined with CVSS + EPSS, risk-sorted
- ✓ **Feed work-queue** — explainable relevance scoring, claim grouping
- ✓ **ATT&CK actor/malware profiles + RELATED** entity index
- ✓ **Collection health**, source registry, trends

PLANNED EVOLUTION (ADDITIVE)

- **AI Daily Brief** — Framework local LLM writes the shift brief (passthrough already wired)
- **Client / sector relevance** — "affects our clients" filter, the MSSP differentiator
- **Wave-2 collectors** — FeodoTracker, C2IntelFeeds, PhishTank, APTnotes
- **Push / alerting** — shift digest to Slack/email
- **Decouple data-refresh from site-deploy** for higher cadence

DESIGN DECISIONS TO LOCK FIRST

- What the feed is *for* (brief vs. relevance-filter vs. hunt-feed)
- Data backbone (static vs. KV/R2) · watchlist location · alerting scope

Approach: a proper design pass before building, on top of the existing pipeline. The early "garbage" was the UI/identity — long since fixed — not the data layer.

SOCDesk stops being only an aggregator and starts producing first-party intelligence: our own honeypot sensors observing real attacks, published as a live feed that loops back into Legs 1 & 2.

CORE FUNCTIONS

- **Own honeypot sensor(s)** — Cowrie-class, capturing live SSH/Telnet/scan activity
- **Live attack telemetry & globe visualisation**
(mockup: `design/mockups/h-sensor.html`)
- **First-party IOC feed** — observed attacker infrastructure, feeding Leg-1 verdicts and Leg-2 feed
- Trend surfacing — emerging scan campaigns, credential-spray patterns

SECURITY NON-NEGOTIABLES

- **No Tailscale on the sensor** — no path from the honeypot into the tailnet
- **Cloud-plane egress deny** by default
- **Publish /24s, not full IPs** — no doxxing of compromised hosts
- **Allowlist published tokens**; rebuild from cloud-init, never snapshots

GATE

- ▶ Resolve the **employer-IP-assignment question** first — it gates this Leg (and the repo-private decision)

SEQUENCING & CURRENT STATUS

- ✓ **Done:** Leg 1 is LIVE at socdesk.io — multi-source enrichment, the consensus tally, live `/api/enrich`. AAA rebuild built in-tree: Vite + React + Tailwind + Motion under strict CSP (phases 1–4 + site effects), a three.js globe hero with a real light mode, per-entity escalation cards, and extension parity (shared card package + own MV3 build, phases 0–2). Spec: `docs/superpowers/specs/2026-08-12-aaa-modern-stack-design.md`.
- ▶ **Now:** Overview board rework (per-lane selection over the score-sort artifact; ransomware-activity leaderboard + named-actor lane; de-LARP voice), then the deploy flip site/ → web/ (gated), then extension final parity QA in Chrome.
- **Then:** settle the employer-IP question → repo public vs. private; Leg 2 threat-intel feed on the existing pipeline (dedup + prioritisation first, then scale article volume).
- **Later:** Leg 3 sensor, once the IP gate clears.

04 HOSTING & CI/CD

AS SHIPPED

Two app trees live in one repo. `site/` is the legacy vanilla ES-module app and is **what serves socdesk.io today**. `web/` is the new Vite + React + TS + Tailwind + Motion "AAA" app — built clean in-tree but **not yet deployed**. The flip is planned, not done.

HOST & DEPLOY METHOD

- ✓ **Cloudflare Pages**, project socdesk, domain socdesk.io
- ✓ **Direct Upload** via cloudflare/wrangler-action@v3 — deliberately *not* the Git integration
- ✓ **Why not Git integration**: free-plan builds are capped at **500/mo**; this workflow deploys **~1,440x/mo** (twice hourly). Direct uploads don't count against the cap
- ✓ **Repo stays private** — Cloudflare never needs read access to the source
- ✓ **Production vs preview routing** keyed on `--branch=${vars.CF_PRODUCTION_BRANCH} || 'main' }`

WORKFLOW · .GITHUB/WORKFLOWS/COLLECT-AND-DEPLOY.YML

- ✓ **Triggers**: `cron 11,41 * * * *`
`workflow_dispatch · push to main touching data/brief.json`
- ✓ **Steps**: `checkout → setup-python 3.12 → pip install → pytest tests/ → python run_pipeline.py → commit data/state + pull-rebase-push → wrangler pages deploy site`
- ✓ **No self-trigger loop**: the state commit uses `GITHUB_TOKEN`, whose pushes can't retrigger Actions
- ✓ **Intentional asymmetry**: the Framework's `brief.json` pushes use a deploy key, so *those* DO trigger this workflow

```
deploy → wrangler pages deploy site --project-name=socdesk --branch=${{
vars.CF_PRODUCTION_BRANCH || 'main' }}
```

The Cloudflare "documented example" `githubToken/deployments-write` path is deliberately omitted — at ~1,440 runs/mo it would file that many GitHub Deployment records. Consistent with `docs/OPERATIONS.md § "Why direct upload rather than the Git integration"`.

05 DATA FLOW & ENRICHMENT

AS SHIPPED

Git is the datastore — one small JSON file per payload, per day. All collectors are keyless/public; the only secrets in the system are the enrichment API keys, held server-side in Cloudflare.

PIPELINE OUTPUT · RUN_PIPELINE.PY

- ✓ `run(...)` writes each payload to **both** `out_dir` and `state_dir`
- ✓ `__main__` pins `out_dir="site/data"`, `state_dir="data/state"`
- ✓ Rolling `data/state/*.json` (+ history/series) are committed each run — last-known-good survives an upstream outage
- ✓ Payloads: `feed · cves · health · actors · malware · relations · sources · trends`; `brief.json` passed through if present (Tier 2)
- ✓ **No IOC corpus published** — reputation data is reached by user-clicked deep links at render, never mirrored (COMPLIANCE.md)

⚠ DATA GAP FOR THE FLIP

- ▶ `web/public/data/state/*.json` is a **SEPARATE** committed copy that the pipeline does **not** refresh — it only writes `site/data + data/state`. Deploy `web/` as-is and its data freezes at whatever was last hand-committed.

ENRICHMENT FUNCTION · /API/ENRICH

- ✓ **Cloudflare Pages Function** at `functions/api/enrich.js` → logic in `lib/enrich.mjs`
- ✓ **Types:** `ipv4 · domain · url · md5 · sha1 · sha256`
- ✓ **Sources:** `AbuseIPDB · VirusTotal · GreyNoise · MalwareBazaar · ipinfo · urlscan`
- ✓ **Same-origin by design** — a Function on the site's own domain keeps CSP `connect-src 'self'`; no `api.*` origin, no CORS layer
- ✓ **Keys in Pages project secrets** (`ABUSEIPDB_API_KEY · VIRUSTOTAL · GREYNOISE_API_KEY · IPINFO_TOKEN · URLSCAN_API_KEY`) — never in git or the browser; MalwareBazaar is keyless
- ✓ **Co-deployed** with the Pages project: `functions/` sits at the repo root, picked up alongside the deployed asset dir

```
collectors (keyless) → run_pipeline.py → site/data/*.json + commit data/state/*.json → wrangler
deploy site → socdesk.io · browser → /api/enrich → AbuseIPDB / VT / GreyNoise /
MalwareBazaar / ipinfo / urlscan
```

CONTENT-SECURITY-POLICY & HEADERS

- ✓ **Strict CSP, no inline anything.** Core across both trees: `default-src 'none'; script-src 'self'; style-src 'self'`
- ✓ **Live (`site/_headers`)** extends it with `img-src 'self' data:,font-src 'self',connect-src 'self',worker/manifest-src 'self',base-uri/form-action/object-src 'none',frame-ancestors 'none',upgrade-insecure-requests` — plus Permissions-Policy, COOP, X-Frame-Options: DENY
- ✓ Keys never reach the browser; collectors carry none. Analyst state is `localStorage` only — no accounts, no analytics

⚠ HEADER PARITY GAP

- ▶ **`web/public/_headers` is looser than the live one:** it omits `frame-ancestors 'none'`, `upgrade-insecure-requests`, Permissions-Policy, COOP and X-Frame-Options. Bring to parity before the flip.

PALETTE DIVERGENCE — INTENTIONAL, UNTIL THE FLIP

- ✓ **`web/` moved to cool-slate tokens** (2026-08-14, `shared/tokens.css`) — the old warm-espresso ground clashed with the periwinkle accent
- ✓ **`site/` + the Copy-card PNG stay WARM** (`site/css/tokens.css` RADAR + `drawVerdict`) — a follow-up retune, deliberately divergent until `site/` is superseded
- ✓ One token source now feeds web app + extension (`shared/tokens.css`) so those two can't drift

Role split held in both: warm/cool neutrals = the room; periwinkle = the product (never a verdict); red/amber/green = verdict & severity meaning only.

What must change to point socdesk.io from the vanilla `site/` at the React `web/` build. None of this is done — the flip is gated on an explicit go.

- **Dual-write fresh data.** The pipeline must also publish to `web/public/data/state` (or the deploy must build from a dir the pipeline refreshes). Today it only refreshes `site/data` — deploy `web/` without this and the live app's data freezes.
- **Add `web/public/_redirects`** with `/* /index.html 200` (excluding `/api/*` and `/data/*`) for SPA deep-link fallback. The React app uses **PATH routing** (`/lookup · /desk · /actor · /gallery`) — unlike `site/`'s hash routing, a hard refresh on a deep path 404s without this.
- **Build step in the workflow.** Add Node setup + `npm ci + vite build` in `web/`, and deploy `web/dist` instead of `site`. Verify the `repo-root` functions/ still co-deploys so `/api/enrich` keeps working.
- **De-risk with a preview.** Push a Cloudflare Pages **preview** (non-production branch) deploy and validate it before flipping production — the live domain is the only current target.

CONCERN	CURRENT STATE	GAP	FIX
Live data	Pipeline writes <code>site/data</code> + commits <code>data/state</code>	<code>web/public/data/state</code> is a stale committed copy the pipeline never refreshes	Dual-write fresh payloads to <code>web/public/data/state</code>
Deep links	<code>web/</code> uses path routing; <code>site/</code> uses hash routing	No <code>web/public/_redirects</code> → hard-refresh on a deep path 404s	Add <code>/* /index.html 200</code> , excluding <code>/api/*</code> & <code>/data/*</code>
Build	Workflow deploys static <code>site/</code> as-is	No Node/ <code>npm ci + vite build</code> step; <code>web/dist</code> not produced in CI	Add Node + <code>npm ci + vite build</code> ; deploy <code>web/dist</code>
Enrich fn	<code>functions/</code> at repo root co-deploys with <code>site</code>	Co-deploy unverified when the asset dir becomes <code>web/dist</code>	Confirm <code>/api/enrich</code> after switching the deploy dir
Header parity	<code>site/_headers</code> adds frame-ancestors, upgrade-insecure-requests, Permissions-Policy, COOP, X-Frame-Options	<code>web/public/_headers</code> omits all of those	Bring <code>web/public/_headers</code> to parity first
Rollout	Production is the only deploy target	Flipping blind risks the live domain	Preview (non-prod branch) deploy & validate before flip